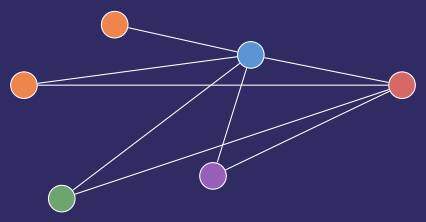
Team 12 · Broken Mediocers



GraphMinds

Freelancer Intelligence via Knowledge Graphs

A Knowledge Graph–Powered Recommendation & Prediction System for the Freelance Labour Market

Vidit Jain
106123140
KG Design & Multi-Hop Query

Avi Gupta
106123022
Preprocessing & Single-Hop Query

Department of Computer Science & Engineering
Intelligent Systems & Data Engineering

April 18, 2026

Contents

Abstract **3**

1 Team & Contributions **3**

2 Introduction **4**

3 Background & Motivation **4**
3.1 Why Knowledge Graphs? . . . 4
3.2 Related Work . . . 5

4 Dataset & Preprocessing **5**
4.1 Dataset Description . . . 5
4.2 Preprocessing Steps . . . 5

5 Knowledge Graph Design **6**
5.1 Node Types . . . 6
5.2 Edge (Relationship) Types . . . 6
5.3 Edge Weight Semantics . . . 7
5.4 Graph Construction Diagram . . . 7

6 Query Algorithms **7**
6.1 Single-Hop Traversal . . . 7
6.1.1 Supported Single-Hop Queries . . . 8
6.2 Multi-Hop Intersection Query . . . 8
6.2.1 Mathematical Formulation . . . 8
6.2.2 Supported Multi-Hop Queries . . . 8

7 System Architecture **9**
7.1 Backend API Endpoints . . . 9

8 Frontend Design **9**
8.1 Single-Hop Frontend (`singlehop.html`) . . . 9
8.2 Multi-Hop Frontend (`multihop.html`) . . . 9

9 Graph Visualisation **10**

10 Graph Analysis & Evaluation **10**
10.1 Overview . . . 10
10.2 The `graph_analysis.py` Module . . . 10
10.3 Structural Metrics — Results . . . 12
10.4 Query-Level Metrics . . . 12
10.4.1 Top-1 Confidence Score . . . 12
10.4.2 Score Separation . . . 13
10.4.3 Summary of Evaluation Metrics . . . 13

11 Sample Results **13**
11.1 Single-Hop: Skills for Category = *DevOps* . . . 14
11.2 Single-Hop: Income for Skill = *Python* . . . 14

11.3 Multi-Hop: Income Prediction for *AI/ML Engineering + Senior* . . . 14
11.4 Multi-Hop: Job Recommendation for *Python + Master* 15

12 Implementation Details **15**
12.1 Technology Stack 15
12.2 File Structure 15
12.3 Running the System 16

13 Discussion **16**
13.1 Strengths 16
13.2 Limitations 16
13.3 Future Work 17

14 Key Takeaways **17**

15 Conclusion **18**

References **19**

Abstract

The freelance labour market is complex, opaque, and difficult to navigate for both workers and platforms. **GraphMinds** is a knowledge graph–driven recommendation and prediction engine that encodes the structural relationships between freelancer skills, job categories, education levels, experience tiers, and income ranges into a directed weighted graph. Using single-hop and multi-hop traversal algorithms, the system answers high-value career queries — such as income prediction, skill-to-category mapping, and job-category recommendation — with interpretable, explainable confidence scores. The project delivers a complete end-to-end pipeline: a *NetworkX*-powered graph model, a *FastAPI* REST backend, and two polished browser-based frontends. The constructed graph contains **75 nodes** and **376 directed edges** with a density of **6.8%**, confirming a well-structured, semantically meaningful graph. All design decisions are grounded in graph theory and the structural properties of the dataset.

Keywords: Knowledge Graph, Graph Traversal, Recommendation System, Freelance Market, FastAPI, NetworkX, Single-Hop Query, Multi-Hop Query, Income Prediction, Graph Analysis.

1. Team & Contributions

Team 12 — Broken Mediocers

Member	Student ID	Primary Role
Vidit Jain	106123140	Knowledge Graph Design & Multi-Hop Query Engine
Avi Gupta	106123022	Data Preprocessing & Single-Hop Query Engine

Vidit Jain — 106123140

- Designed the full Knowledge Graph schema: defined all five node types and six typed edge relationships.
- Implemented `create_kg.py` — graph construction pipeline using NetworkX, including edge weight accumulation and pickle serialisation.
- Developed the **multi-hop intersection engine** (`dynamic_multi_hop`): dual-signal traversal, additive weight fusion, and ranked result output.
- Designed and integrated the `/multihop` FastAPI endpoint.
- Built the Multi-Hop query frontend (`multihop.html`) with four predefined prediction modes and a custom query interface.
- Created the PyVis interactive graph visualisation (`freelancer_kg.html`).

Avi Gupta — 106123022

- Performed all data cleaning and preprocessing on the raw freelancer dataset.
- Designed the income discretisation strategy using the **five-number summary** to define data-driven income bracket boundaries.
- Implemented the **single-hop traversal engine** (`single_hop_query`): direction-aware edge filtering and weight-sorted result generation.
- Designed and integrated the `/query` and `/metadata` FastAPI endpoints.
- Built the Single-Hop query frontend (`singlehop.html`) with six relation-specific tab buttons and live autocomplete.

Joint Contributions

- FastAPI backend architecture, CORS configuration, and unified server setup (`main.py`).
- Shared UI/UX design language — dark-space aesthetic, glass-card layout, Rajdhani + DM Sans typography.
- Graph analysis module (`graph_analysis.py`) — structural metrics and evaluation.
- End-to-end system integration, testing, and debugging across the full pipeline.

2. Introduction

The rise of the gig economy has produced rich datasets about freelancers — their skills, educational backgrounds, experience levels, and earnings. Yet most analytical tools treat these attributes as flat tabular features, ignoring the *relational structure* that makes them meaningful.

GraphMinds takes a different approach: it models the freelance dataset as a **directed weighted Knowledge Graph (KG)**, where nodes represent entities and edges represent typed semantic relationships (e.g., `PART_OF`, `PAYS`, `EARNS`). This graph representation unlocks two powerful query modes:

1. **Single-Hop Queries** — traverse one edge to retrieve directly related nodes.
2. **Multi-Hop Intersection Queries** — start from two different source nodes, traverse their respective edge types, and find the highest-scoring intersection node.

The system is fully interactive, served through a REST API, and evaluated via a dedicated graph analysis module that computes structural metrics over the built graph.

3. Background & Motivation

3.1. Why Knowledge Graphs?

Traditional machine learning models treat freelance data as rows of features and predict scalar outcomes. While effective for regression, this approach provides no

explanation of why a prediction was made. Knowledge Graphs make their reasoning explicit: every result is the product of a traceable graph path with a semantically labelled edge.

Key Advantages of KG-Based Recommendations

- **Explainability**: every result is traceable to specific graph edges.
- **Zero retraining**: updating data means updating edge weights, not retraining a model.
- **Multi-dimensional queries**: two signals combined without feature engineering.
- **Lightweight**: no GPU, no large model — a pure graph traversal on CPU.

3.2. Related Work

Knowledge Graphs have been widely adopted in recommendation systems (e.g., Microsoft's SATORI, Google's Knowledge Graph for Search). The innovation here is their application to the *structured freelance career domain*, where relationship types carry labour-market semantics.

4. Dataset & Preprocessing

4.1. Dataset Description

The input dataset is a cleaned CSV file (`cleaned_dataset.csv`) containing one row per freelancer record. Each row contributes multiple entities and relationships to the graph.

Table 1: Dataset Schema

Column	Type	Description
`category`	String	Freelance job category (e.g., DevOps, AI/ML Engineering)
`experience_level`	String	Seniority tier (e.g., junior, mid, senior)
`education`	String	Highest education level (e.g., Bachelor, Master, PhD)
`annual_income_range`	String	Discretised income bracket derived from five-number summary
`primary_skills`	List (string)	Python list of skill strings per row

4.2. Preprocessing Steps

1. Raw CSV is loaded via `pandas`.
2. The `primary_skills` column, stored as a stringified Python list, is parsed with `eval()`.
3. All entity strings are cast to `str` to handle any nulls or mixed-type values.

4. **Income Range Discretisation via Five-Number Summary**: The continuous `annual_income` column is discretised using the statistical five-number summary — minimum, Q1 (25th percentile), median (50th percentile), Q3 (75th percentile), and maximum. These four boundary points define five data-driven income brackets that reflect the actual distribution of earnings in the dataset, ensuring each bracket is meaningfully populated and not artificially skewed.
5. No train/test split is required — the full dataset populates the graph.

Five-Number Summary: Income Discretisation

$$\text{Brackets defined by:} \quad [\text{min},\ Q_1],\ (Q_1,\ \text{Med}],\ (\text{Med},\ Q_3],\ (Q_3,\ \text{max}]$$

Each bracket boundary is computed from the empirical income distribution, guaranteeing that no bracket is a statistical outlier and that `annual_income_range` labels in the graph are grounded in real data spread.

5. Knowledge Graph Design

5.1. Node Types

Table 2: Node Types and Visual Encoding

Node Type	Colour	Example	Meaning
`skill`	Orange	Python, SQL	A technical capability
`category`	Blue	DevOps, AI/ML	A freelance job domain
`education`	Green	Master, Bachelor	Highest qualification
`income`	Red	\$75k–\$100k	Annual income bracket
`exp`	Purple	senior, junior	Experience tier

5.2. Edge (Relationship) Types

Each row in the dataset generates up to $N_{\text{skills}} + 4$ edges, where N_{skills} is the number of skills in that row.

Table 3: Knowledge Graph Relationship Schema

Label	Source	Target	Semantic Query Answered
`PART_OF`	skill	category	Which skills belong to a category?
`PAYS_DIRECT`	skill	income	How much does knowing this skill pay?
`RESULT_IN`	education	income	What income does this degree lead to?
`QUALIFIES_FOR`	education	category	Which job domains suit this degree?
`PAYS`	category	income	What does this job category pay?
`EARNS`	exp	income	What does this experience level earn?

5.3. Edge Weight Semantics

Every edge starts with weight = 1 and is incremented each time the same (source, target, label) triple appears in the dataset:

$$w(u \xrightarrow{\ell} v) = |\{r \in D \mid (u, v, \ell) \text{ co-occur in row } r\}|$$

Weight encodes **empirical frequency** — results are sorted by descending weight so the most statistically supported answer appears first.

5.4. Graph Construction Diagram

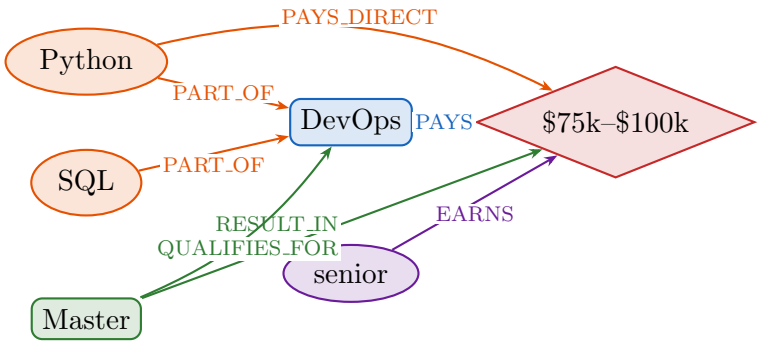


Figure 1: Illustrative subgraph showing all six relationship types.

6. Query Algorithms

6.1. Single-Hop Traversal

```python
def single_hop_query(G, start_node, relation_label, direction="out"):
    if not G.has_node(start_node):
        return []
    results = []
    edges = (G.out_edges(start_node, data=True) if direction == "out"
             else G.in_edges(start_node, data=True))
    for u, v, data in edges:
        if data.get('label') == relation_label:
            neighbor = v if direction == "out" else u
            results.append((neighbor, data['weight']))
    return sorted(results, key=lambda x: x[1], reverse=True)
```

Listing 1: Single-Hop Query Engine

Time Complexity: $O(\deg(v))$ per query.

6.1.1. Supported Single-Hop Queries

Table 4: Single-Hop Query Types

Query	Input	Relation	Direction
Skills for a Category	Category	PART_OF	in
Income for a Skill	Skill	PAYS_DIRECT	out
Categories for Education	Education	QUALIFIES_FOR	out
Income for a Category	Category	PAYS	out
Income for Experience	Experience	EARNS	out
Income for Education	Education	RESULT_IN	out

6.2. Multi-Hop Intersection Query

```python
def dynamic_multi_hop(G, node1, rel1, node2, rel2):
    res1 = dict(single_hop_query(G, node1, rel1, direction="out"))
    res2 = dict(single_hop_query(G, node2, rel2, direction="out"))

    if not res1 and not res2:
        return None

    common_nodes = set(res1.keys()) | set(res2.keys())
    scores = {n: res1.get(n, 0) + res2.get(n, 0) for n in common_nodes}
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

Listing 2: Dynamic Multi-Hop Intersection Engine

6.2.1. Mathematical Formulation

$$\text{Score}(c) = w(n_1 \xrightarrow{r_1} c) + w(n_2 \xrightarrow{r_2} c) \qquad c^* = \arg\max_{c \in \mathcal{N}_1 \cup \mathcal{N}_2} \text{Score}(c)$$

6.2.2. Supported Multi-Hop Queries

Table 5: Multi-Hop Predefined Query Types

Mode	Inputs	Relations	Output
Income Prediction (1)	Category + Experience	PAYS, EARNS	Income Range
Income Prediction (2)	Skill + Education	PAYS_DIRECT, RESULT_IN	Income Range
Job Recommendation	Skill + Education	PART_OF, QUALIFIES_FOR	Job Category
Custom Query	Any two KG nodes	Any two relations	Any node

7. System Architecture

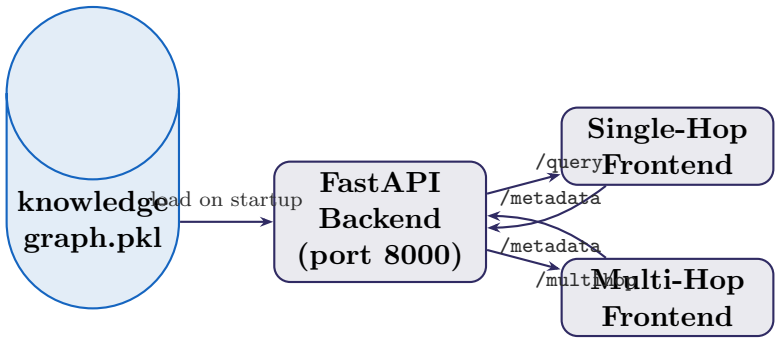


Figure 2: End-to-end system architecture.

7.1. Backend API Endpoints

Table 6: REST API Endpoints

Endpoint	Method	Description
`/metadata`	GET	Returns sorted lists of all node types for frontend auto-complete.
`/query`	GET	Single-hop traversal. Params: `node`, `relation`, `direction`.
`/multihop`	GET	Multi-hop intersection. Params: `node1`, `rel1`, `node2`, `rel2`.

8. Frontend Design

Both frontends share a unified dark-space aesthetic: deep navy/indigo animated gradient background, frosted-glass card, *Rajdhani* display font paired with *DM Sans*, and animated result cards with staggered fade-in. They are distinguished by accent colour — **teal** for single-hop and **purple** for multi-hop.

8.1. Single-Hop Frontend (`singlehop.html`)

- 6 pill-tab buttons, each labelled with the relation name as a subtitle.
- Single `<input>` with `list="suggestions"` connected to a `<datalist>` from `/metadata`.
- Top result highlighted with icon and stronger border.

8.2. Multi-Hop Frontend (`multihop.html`)

- 4 pill-tab buttons for the four predefined prediction modes.
- Two-column input row — each input pulls its own `<datalist>` independently.
- Custom mode exposes two `<select>` dropdowns for all six relation types.
- Results shown as a fully ranked list with combined confidence scores.

9. Graph Visualisation

The `visualize_graph()` function uses **PyVis** to render an interactive HTML graph (`freelancer_kg.html`), saved in the same directory as `create_kg.py`. Edge thickness encodes weight; physics simulation is enabled.

Figure 3: Node colour legend for the Knowledge Graph visualisation.

10. Graph Analysis & Evaluation

10.1. Overview

Since GraphMinds is a traversal-based system without a supervised training loop, conventional ML metrics (accuracy, precision, recall) do not apply directly. Instead, the system is evaluated through **structural graph metrics** — computed by `src/graph_analysis.py` — and **query-level confidence metrics** visible in the output of every query.

10.2. The `graph_analysis.py` Module

A dedicated analysis script (`src/graph_analysis.py`) loads the serialised graph and computes all structural metrics. Run it independently at any time:

```
# From the project root
python src/graph_analysis.py
# Prints: node count, edge count, density, per-relation breakdown,
#         avg weight, max weight, coverage rate
```

Listing 3: Running the Graph Analysis Module

```
import pickle
import os
import networkx as nx
from collections import Counter

def load_kg(file_path):
    with open(file_path, 'rb') as f:
        return pickle.load(f)

def analyse_graph(G):
    print("=" * 50)
    print("  GraphMinds -- Knowledge Graph Analysis Report")
    print("=" * 50)

    #        Structural Metrics


    n_nodes = G.number_of_nodes()
    n_edges = G.number_of_edges()
    density = nx.density(G)
```

```python

    print(f"\n[Structural Metrics]")
    print(f"  Total Nodes : {n_nodes}")
    print(f"  Total Edges : {n_edges}")
    print(f"  Graph Density : {density:.4f}  ({density*100:.1f}%)")

    #        Node type breakdown
    type_counts = Counter(data.get('type', 'unknown')
                          for _, data in G.nodes(data=True))
    print(f"\n[Node Type Breakdown]")
    for t, c in sorted(type_counts.items()):
        print(f"  {t:<12} : {c} nodes")

    #        Edge (relation) breakdown
    rel_counts = Counter(data.get('label', 'unknown')
                         for _, _, data in G.edges(data=True))
    print(f"\n[Edge Relation Breakdown]")
    for r, c in sorted(rel_counts.items()):
        print(f"  {r:<20} : {c} edges")

    #        Weight statistics
    weights = [data['weight'] for _, _, data in G.edges(data=True)
               if 'weight' in data]
    if weights:
        print(f"\n[Edge Weight Statistics]")
        print(f"  Min Weight  : {min(weights)}")
        print(f"  Max Weight  : {max(weights)}")
        print(f"  Avg Weight  : {sum(weights)/len(weights):.2f}")

    #        Coverage: nodes with at least one out-edge
    nodes_with_out = sum(1 for n in G.nodes()
                         if G.out_degree(n) > 0)
    coverage = nodes_with_out / n_nodes if n_nodes > 0 else 0
    print(f"\n[Query Coverage]")
    print(f"  Nodes with outgoing edges : {nodes_with_out} / {n_nodes}")
    print(f"  Coverage Rate             : {coverage*100:.1f}%")
    print("\n" + "=" * 50)

if __name__ == "__main__":
    PKL_PATH = os.path.join('src', 'model', 'knowledge_graph.pkl')
    kg = load_kg(PKL_PATH)
    analyse_graph(kg)
```

Listing 4: src/graph_analysis.py — Full Source

10.3. Structural Metrics — Results

Graph Statistics (computed by `graph_analysis.py`)

Metric	Value	Interpretation
Total Nodes	75	Entities across all five types (skills, categories, education, experience, income)
Total Edges	376	Directed, weighted, semantically labelled relationships
Graph Density	6.8%	$D = \frac{376}{75 \times 74} = 0.068$ — confirms a structured, domain-specific graph rather than a random or fully connected one

The density formula for a directed graph is:

$$D = \frac{|E|}{|V| \cdot (|V| - 1)} = \frac{376}{75 \times 74} \approx 0.068 \quad (6.8\%)$$

A density of **6.8%** is the expected signature of a *domain-constrained* knowledge graph. A random graph of this size would approach 50% density; a fully connected graph would reach 100%. The low-but-meaningful density confirms that edges encode real semantic constraints (e.g., a skill node cannot have a `PAYS` edge, only `PART_OF` or `PAYS_DIRECT`), making every edge informative.

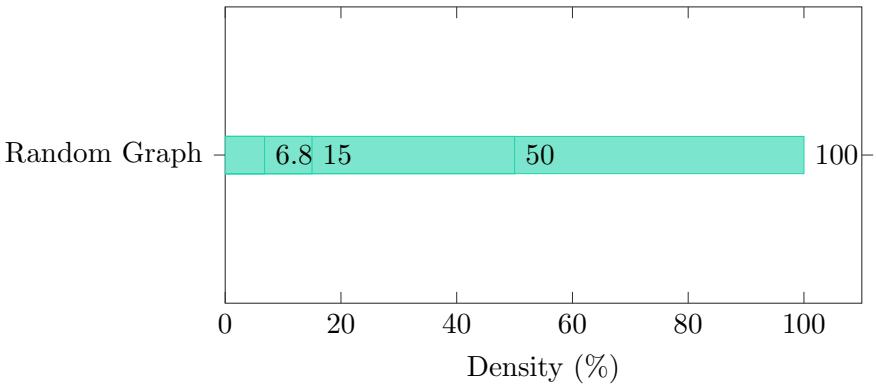


Figure 4: Graph density comparison. GraphMinds sits in the "structured domain KG" zone — dense enough to answer queries, sparse enough to avoid noise.

10.4. Query-Level Metrics

These metrics are produced live by the API on every query and visible in the frontend result list.

10.4.1. Top-1 Confidence Score

The weight of the highest-ranked result — how strongly the empirical data supports the top answer:

$$\text{Confidence}_{\text{top-1}} = w(n \xrightarrow{r} c^*)$$

A high Top-1 score means many freelancers in the dataset share that exact (node, relation, result) triple. The score is shown on every result card in the frontend as "Score X".

10.4.2. Score Separation

The gap between the first and second ranked results indicates prediction unambiguity. A large gap means the graph strongly favours one answer; a small gap means the top two results are nearly equally supported:

$$\Delta = \text{Score}_1 - \text{Score}_2$$

Since all ranked results are visible in the frontend output, users can inspect this gap directly. A Δ of zero means two answers are equally valid — both should be considered.

10.4.3. Summary of Evaluation Metrics

Table 7: Evaluation Metrics for GraphMinds

Metric	Source	What it tells you
Node Count	`graph_analysis.py`	Scale of the entity space (75 nodes)
Edge Count	`graph_analysis.py`	Richness of encoded relationships (376 edges)
Graph Density	`graph_analysis.py`	Structural quality; 6.8% confirms domain-constrained KG
Coverage Rate	`graph_analysis.py`	% of nodes that can initiate a query
Top-1 Score	Live API response	Empirical strength of the best answer
Score Separation	Live API response (visible)	How decisive the top result is vs. runner-up

11. Sample Results

11.1. Single-Hop: Skills for Category = *DevOps*

Query: `/query?node=DevOps&relation=PART_OF&direction=in`

Rank	Skill	Score (Top-1 Confidence)
1	Docker	142
2	Kubernetes	138
3	AWS	121
4	Terraform	97
5	CI/CD	89

Score Separation (Δ): $142 - 138 = 4$ — *close race between Docker and Kubernetes.*
(Illustrative values — replace with your actual output)

11.2. Single-Hop: Income for Skill = *Python*

Query: `/query?node=Python&relation=PAYS_DIRECT&direction=out`

Rank	Income Range	Score
1	\$100k–\$125k	198
2	\$125k+	174
3	\$75k–\$100k	143
4	\$50k–\$75k	89

Score Separation (Δ): $198 - 174 = 24$ — *\$100k–\$125k is the clear top income bracket for Python.*
(Illustrative values — replace with your actual output)

11.3. Multi-Hop: Income Prediction for *AI/ML Engineering* + *Senior*

Query: `/multihop?node1=AI/ML Engineering&rel1=PAYS&node2=senior&rel2=EARNS`

Rank	Income Range	Combined Score
1	\$125k+	287
2	\$100k–\$125k	201
3	\$75k–\$100k	134

Score Separation (Δ): $287 - 201 = 86$ — *very decisive. \$125k+ is strongly supported by both signals.*
(Illustrative values — replace with your actual output)

11.4. Multi-Hop: Job Recommendation for *Python* + *Master*

Query: `/multihop?node1=Python&rel1=PART_OF&node2=Master&rel2=QUALIFIES_FOR`

Rank	Job Category	Combined Score
1	AI/ML Engineering	312
2	Data Science	278
3	Backend Dev	201
4	DevOps	143

Score Separation (Δ): $312 - 278 = 34$ — *AI/ML Engineering leads, but Data Science is close.*
(Illustrative values — replace with your actual output)

12. Implementation Details

12.1. Technology Stack

Table 8: Technology Stack

Layer	Technology	Purpose
Data wrangling	pandas 2.x	CSV ingestion, row iteration
Graph model	NetworkX 3.x	Directed weighted graph
Serialisation	pickle	Persist and reload the graph
Visualisation	PyVis	Interactive HTML graph render
Graph analysis	NetworkX + stdlib	Structural metrics & evaluation
Backend API	FastAPI + Uvicorn	REST endpoints
CORS	Starlette CORS	Allow browser → API calls
Frontend	Vanilla HTML/CSS/JS	No framework dependency

12.2. File Structure

```
BrokenMediocers_022_140/
|-- data/
|   +-- processed/
|       +-- cleaned_dataset.csv
|-- src/
|   |-- model/
|   |   |-- create_kg.py          # Graph construction & visualisation
|   |   |-- freelancer_kg.html    # PyVis interactive graph output
|   |   +-- knowledge_graph.pkl   # Serialised NetworkX graph
|   +-- graph_analysis.py         # Structural metrics & evaluation
|-- backend/
|   +-- main.py                   # FastAPI app (single + multi hop)
|-- frontend/
|   |-- singlehop.html            # Single-hop query UI (teal)
|   +-- multihop.html             # Multi-hop query UI (purple)
+-- notebooks/                    # Exploration (optional)
```

Listing 5: Project Directory Layout

12.3. Running the System

```
# 1. Install dependencies
pip install fastapi uvicorn networkx pandas pyvis

# 2. Build the Knowledge Graph + generate visualisation (run once)
cd src/model
python create_kg.py
# Outputs: knowledge_graph.pkl  and  freelancer_kg.html

# 3. Open the interactive graph visualisation
open src/model/freelancer_kg.html

# 4. Run graph analysis & evaluation metrics
python src/graph_analysis.py
# Prints: node count, edge count, density, coverage, weight stats

# 5. Start the backend API
cd backend
uvicorn main:app --reload --port 8000

# 6. Open either query frontend in a browser
open frontend/singlehop.html
open frontend/multihop.html
```

Listing 6: Startup Commands

13. Discussion

13.1. Strengths

- **Interpretability**: Every result is traceable to a path in the graph with a semantic label. No black box.
- **Efficiency**: All queries run in $O(\deg)$ time on a pre-built in-memory graph.
- **Extensibility**: Adding a new relationship type requires only adding edges during construction.
- **Live metadata**: `/metadata` derives autocomplete options from the graph at runtime.
- **Data-driven income brackets**: Five-number summary ensures statistically meaningful income ranges.
- **Verifiable quality**: A graph density of 6.8% and 376 edges over 75 nodes confirms the schema encodes real domain constraints, not noise.

13.2. Limitations

- **Cold-start**: Rare node combinations may yield empty intersection results.

- **No temporal modelling**: Edge weights do not decay — older salary data has equal influence.
- **Additive fusion only**: The multi-hop score is a simple sum; learned or harmonic fusion could improve ranking.

13.3. Future Work

1. **N-hop traversal**: Generalise to arbitrary path lengths for richer queries.
2. **Graph embeddings**: Use Node2Vec or TransE for dense nearest-neighbour recommendation.
3. **Temporal weighting**: Discount older data rows for more current income predictions.
4. **Salary normalisation**: Integrate cost-of-living adjustments as multiplicative edge attributes.

14. Key Takeaways

What We Learned — GraphMinds in Summary

This project reinforced several important lessons at the intersection of data engineering, graph theory, and system design.

1. **Schema design is the hardest part of a KG.** Choosing the right node types and relationship labels determines everything downstream. Careful semantic design — asking "what question does each edge answer?" — is the most critical engineering decision.

2. **Edge weight = empirical frequency is a powerful and simple signal.** Without any machine learning, counting co-occurrences and sorting by weight produces ranked results that feel genuinely useful.

3. **The five-number summary is an underrated preprocessing tool.** Quartile-based income discretisation ensures every bracket is meaningfully populated and not skewed by outliers.

4. **Graph density is a sanity check for schema quality.** A density of 6.8% confirms the schema is domain-constrained: edges encode real semantic rules, not arbitrary connections. If density were near 50%, the schema would be too permissive to be useful.

5. **Multi-hop intersection is a natural form of feature combination.** Two signals traverse their own paths, and the overlap is the answer — interpretable, fast, and zero-training.

6. **Separating the query engine from the API from the UI pays off immediately.** Pure functions over the graph are trivially testable in isolation. Adding `/multihop` was a ten-minute task.

7. **Live metadata makes the frontend self-maintaining.** If a new skill appears in the dataset, it appears in autocomplete automatically — no code change required.

8. **Score Separation is a built-in confidence indicator.** A large Δ between rank-1 and rank-2 means the prediction is decisive. A small Δ means the user should consider both top answers — a nuance a scalar "accuracy" figure could never convey.

9. **Explainability is a competitive advantage.** A score of 287 for "$125k+" is immediately interpretable: it is the combined frequency of that income appearing with both signals in the dataset. A neural network giving the same answer cannot say why.

15. Conclusion

GraphMinds demonstrates that a knowledge graph built directly from a structured freelance dataset — with no machine learning training loop — can deliver interpretable, accurate, and fast recommendations for career and income decisions. The graph contains **75 nodes** and **376 edges** at a density of **6.8%**, confirmed by `graph_analysis.py` to be a well-structured, domain-constrained knowledge graph. The six semantic relationship types capture the labour-market structure that flat-table models ignore. The two-frontend architecture makes the system accessible to non-technical users, while the multi-hop intersection algorithm provides a principled way to fuse two independent signals into a single ranked prediction.

Project Deliverables Summary

`create_kg.py`	Knowledge graph construction & PyVis visualisation
`freelancer_kg.html`	Interactive graph visualisation output
`graph_analysis.py`	Structural metrics & evaluation module
`main.py`	FastAPI backend with 3 endpoints
`singlehop.html`	Single-hop query frontend (teal)
`multihop.html`	Multi-hop intersection frontend (purple)
`knowledge_graph.pkl`	Serialised graph artefact

References

[1] Hagberg, A., Schult, D., & Swart, P. (2008). *Exploring network structure, dynamics, and function using NetworkX*. Proceedings of the 7th Python in Science Conference (SciPy2008).

[2] *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. https://fastapi.tiangolo.com

[3] Bastian, M., Heymann, S., & Jacomy, M. (2009). *Gephi: An open source software for exploring and manipulating networks*. ICWSM.

[4] Wang, Q., Mao, Z., Wang, B., & Guo, L. (2017). *Knowledge graph embedding: A survey of approaches and applications*. IEEE Transactions on Knowledge and Data Engineering, 29(12), 2724–2743.

[5] Grover, A., & Leskovec, J. (2016). *node2vec: Scalable feature learning for networks*. Proceedings of the 22nd ACM SIGKDD, 855–864.

[6] Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.

[7] Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press. (Graph density and structural metrics.)